



中国海洋大学
OCEAN UNIVERSITY OF CHINA

系统开发工具基础实验报告

第 4 周：代码质量；元编程；大杂烩；课堂综合测试

姓名： 姬艺华

学号： 25020007046

日期： 2026 年 9 月 11 日

目录

1	课上习题	3
1.1	第 13 题：建立可执行的本地质量门禁	3
1.1.1	任务说明	3
1.1.2	关键操作	3
1.1.3	实验结果	4
1.2	第 14 题：让 Make 只重建真正受影响的产物	5
1.2.1	任务说明	5
1.2.2	关键操作	5
1.2.3	实验结果	6
1.3	第 15 题：把本地 API 数据转换为可读报告	7
1.3.1	任务说明	7
1.3.2	关键操作	7
1.3.3	实验结果	9
1.4	第 16 题：修复并交付一个陌生的小型工具仓库	9
1.4.1	任务说明	10
1.4.2	关键操作	10
1.4.3	实验结果	11
2	课下习题	12
2.1	Python 命令行参数练习	12
2.2	pytest 测试字符串功能	13
2.3	使用 ruff 检查代码	13
2.4	使用 Makefile 管理 Python 程序	14
2.5	理解 Makefile 依赖关系	14
2.6	使用 curl 访问本地服务	14
2.7	使用 jq 统计 JSON 数据	15
2.8	使用 jq 过滤数据	15
2.9	循环执行程序直到失败	16
2.10	目录文件数量统计脚本	18
2.11	文本文件单词数量统计脚本	19
3	Git 与作业提交	20
3.1	仓库内容整理	20
3.2	GitHub / Gitee 提交记录	20
3.3	commit 记录截图、仓库页面截图	20

4 问题与总结	21
4.1 遇到的问题	21
4.2 实验总结	22

1 课上习题

本模块记录上课实验四道例题操作基本过程，以及关键点。

1.1 第 13 题：建立可执行的本地质量门禁

主题：代码质量

1.1.1 任务说明

在 pyproject.toml 中加入 ruff 和 pytest 配置，补充正常姓名和空白姓名测试，运行 ruff format、ruff check、pytest，并编写 check.sh 一键执行检查。

1.1.2 关键操作

加入 ruff 以及 pytest 的最小配置；补充一个正常姓名测试和一个空白姓名测试，每个测试包有效断言；

```
1 #pyproject.toml
2 [tool.ruff]
3 line-length = 88
4 [tool.pytest.ini_options]
5 testpaths = [
6     "tests"
7 ]
8
9 #cli.py
10 import argparse
11 def main():
12     p = argparse.ArgumentParser()
13     p.add_argument("--name", required=True)
14     a = p.parse_args()
15     if not a.name.strip():
16         raise SystemExit(2)
17     print(f"Hello, {a.name}!")
18 if __name__ == "__main__":
19     main()
20
21 #test_cli.py
22 import pytest
23 from greetlab.cli import main
24 def test_empty_name(monkeypatch):
25     monkeypatch.setattr(
26         "sys.argv",
27         ["sdt-greet", "--name", ""]
28     )
29     with pytest.raises(SystemExit):
```

```
30      main()
```

编写 check.sh，依次执行 ruffformat-check、ruff check 和 pytest；运行并保证返回 0：

```
1  #!/usr/bin/env bash
2  set -e
3  ruff format --check
4  ruff check
5  pytest
```

运行 ruffformat、ruff check 和 pytest，修复全部问题：

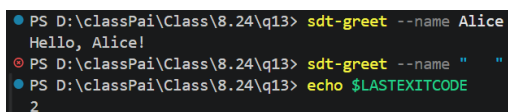
```
chmod +x check.sh
ls -l check.sh

ruff format ruff自动调整书写规范
ruff check 检查代码质量问题
ruff format
./check.sh
```

关键点：

- `ruff format`：自动规范 Python 代码格式，`--check` 参数只检查不修改。
- `ruff check`：静态检查代码问题，例如未使用变量、代码规范错误等。
- `pytest`：自动执行测试用例，通过断言验证程序行为是否符合预期。
- `SystemExit(2)`：用于表示空白姓名属于非法输入，测试时需要检查退出码。
- `capsys` 可以捕获程序输出，用于验证正常输入时是否输出正确问候语。
- `check.sh` 使用 `set -e`，保证任意一步检查失败后立即停止执行。

1.1.3 实验结果



```
PS D:\classPai\Class\8.24\q13> sdt-greet --name Alice
Hello, Alice!
PS D:\classPai\Class\8.24\q13> sdt-greet --name " "
PS D:\classPai\Class\8.24\q13> echo $LASTEXITCODE
2
```

图 1 测试正常姓名以及空白姓名

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q13 (main)
$ ruff format
2 files reformatted, 1 file left unchanged

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q13 (main)
$ ruff format --check
3 files already formatted

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q13 (main)
$ ruff check
All checks passed!

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q13 (main)
$ ruff format
3 files left unchanged

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q13 (main)
$ ./check.sh
3 files already formatted
All checks passed!

===== test session starts =====
platform win32 -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\classPai\Class\8.24\q13
configfile: pytest.ini
plugins: Faker-37.12.0, cov-5.0.0, django-4.11.1
collected 2 items

tests\test_cli.py .. [100%]

===== 2 passed in 0.12s =====
```

图 2 check.sh 脚本自动修改代码问题

1.2 第 14 题：让 Make 只重建真正受影响的产物

主题：元编程；构建系统

1.2.1 任务说明

使用 Makefile 管理一个简单的数据处理流程，让 Make 自动判断：哪些文件已经生成或变化，哪些目标需要重新执行。

1.2.2 关键操作

创建 Makefile：

```
1 .PHONY: all clean
2
3 all: report.txt
4
5 stats.txt: data.csv stats.py
6     python stats.py
7
8 report.txt: report.md stats.txt build_report.py
9     python build_report.py
10
11 clean:
12     rm -f stats.txt report.txt
```

解释代码：

- `.PHONY: all clean`：声明 `all` 和 `clean` 为伪目标（Phony Target）。因为 `all` 和 `clean` 不是实际生成的文件；它们只是用于执行构建任务的命令入口。通过声明为 `.PHONY`，可以避免 Make 将它们误认为文件，从而保证 `make clean` 都会执行清理操作。
- `all: report.txt`：定义默认构建目标。执行 `make` 时，Make 默认执行第一个目标 `all`。由于 `all` 依赖 `report.txt` 因此 Make 会检查并生成最终报告。

- `stats.txt`: `data.csv stats.py python stats.py` : 定义 `stats.txt` 的生成规则; 其中依赖文件: `data.csv`, `stats.py`; 目标文件: `stats.txt`。
- `report.txt`: `report.md stats.txt build_report.py python build_report.py` : 定义最终报告文件的生成规则, 生成 `report.txt` 需要 `report.md`, `stats.txt`, `build_report.py`。
- `clean`: `rm -f stats.txt report.txt` : 执行 `make clean` 会删除由构建过程生成的文件 `stats.txt` 和 `report.txt`。

关键点:

- Make 是一种自动化构建工具, 通过定义目标文件 (target)、依赖文件 (dependency) 和生成命令, 实现自动判断是否需要重新构建。
- 第一次执行 `make` 时, Make 根据依赖关系依次执行 `stats.py` 和 `build_report.py`, 生成 `stats.txt` 和 `report.txt`。
- 第二次执行 `make` 时, 由于源文件未发生变化, 所有目标文件均为最新状态, 因此无需重新执行构建命令。
- 当源文件未修改时, 再次执行 `make` 不会重复执行命令, 提高构建效率。
- Makefile 中命令行前必须使用 Tab 缩进, 不能使用普通空格, 否则会出现 `missing separator` 错误。

1.2.3 实验结果

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q14 (main)
$ make --version
GNU Make 4.4.1
Built for Windows32
Copyright (C) 1988-2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q14 (main)
$ make
python stats.py
python build_report.py

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q14 (main)
$ ls
Makefile build_report.py data.csv report.md report.txt stats.py stats.txt

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q14 (main)
$ make
make: Nothing to be done for 'all'.

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q14 (main)
$ make clean
rm -f stats.txt report.txt

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q14 (main)
$ ls
Makefile build_report.py data.csv report.md stats.py

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q14 (main)
$ make
python stats.py
python build_report.py

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q14 (main)
$ ls
Makefile build_report.py data.csv report.md report.txt stats.py stats.txt

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q14 (main)
$ touch data.csv

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q14 (main)
$ make
python stats.py
python build_report.py
```

图 3 验证实验

1.3 第 15 题：把本地 API 数据转换为可读报告

主题：大杂烩：API、jq、CLI 约定与 Markdown

1.3.1 任务说明

创建本地 JSON 数据文件，并通过 Python HTTP 服务模拟 API 接口；使用 `curl` 获取 HTTP 返回的数据；使用 `jq` 对 JSON 数据进行筛选和排序；编写 `api_report.sh` 自动生成 Markdown 格式的数据报告。

1.3.2 关键操作

运行 `python-mhttp.server 8000`，使用 `curl-fsS` 获取 `http://127.0.0.1:8000/packages.json`；

```
1 python -m http.server 8000
2 打开另一个窗口，测试 jq 是否能读取JSON：
3 curl http://127.0.0.1:8000/packages.json
```

使用 `jq` 筛选 `status` 为 `active` 且 `downloads` 不少于 100 的记录，并按 `downloads` 降序、`name` 升序排列；编写 `api_report.sh`，把结果生成 `summary.md`，包含标题和 `name/version/downloads` 三列表格：

```
1 #测试筛选功能
2 curl http://127.0.0.1:8000/packages.json | jq '
3  .[]
4  | select(.status=="active" and .downloads>=100)
5  '
6 #加排序
7 curl http://127.0.0.1:8000/packages.json | jq '
8  [
9    .[]
10   | select(.status=="active" and .downloads>=100)
11 ]
12 | sort_by(-.downloads, .name)
13 '
```

编写 `api_report.sh`，把结果生成 `summary.md`，包含标题和 `name/version/downloads` 三列表格：

```
1 #api_report.sh
2
3 #!/usr/bin/env bash
4
5 curl -fsS http://127.0.0.1:8000/packages.json |
6 jq -r '
7  [
8    .[]
```

```
9 | select(.status=="active" and .downloads>=100)
10 ]
11 | sort_by(-.downloads, .name)
12 |
13 (
14 "# Package Summary\n\n" +
15 "| name | version | downloads |\n" +
16 "|---|---|---|\n" +
17 (map("| \\.name) | \\.version) | \\.downloads) |") | join("\n"))
18 )
19 ' > summary.md
```

关键点：

- `curl` 用于从本地 HTTP 服务获取 JSON 数据。
- `jq` 用于处理结构化 JSON 数据，相当于 JSON 数据处理工具。
- `select()` 用于根据条件筛选对象。
- `sort_by()` 用于按照指定字段排序。
- 管道符 `|` 将前一个命令的输出传递给下一个命令，实现数据处理流程自动化。

1.3.3 实验结果

```

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q15 (main)
$ curl http://127.0.0.1:8000/packages.json | jq '
[
  | select(.status=="active" and .downloads>=100)
]
'
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
   Dload  Upload   Total             Spent    Left     Speed
100    510  100    510    0      0  42166      0      0      0      0
[
  {
    "name": "alpha",
    "status": "active",
    "downloads": 120,
    "version": "1.2.0"
  },
  {
    "name": "gamma",
    "status": "active",
    "downloads": 450,
    "version": "1.5.1"
  },
  {
    "name": "delta",
    "status": "active",
    "downloads": 450,
    "version": "0.9.0"
  }
]

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q15 (main)
$ curl http://127.0.0.1:8000/packages.json | jq '
[
  | select(.status=="active" and .downloads>=100)
  | sort_by(-.downloads, .name)
]
'
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
   Dload  Upload   Total             Spent    Left     Speed
100    510  100    510    0      0  46982      0      0      0      0
[
  {
    "name": "delta",
    "status": "active",
    "downloads": 450,
    "version": "0.9.0"
  },
  {
    "name": "gamma",
    "status": "active",
    "downloads": 450,
    "version": "1.5.1"
  },
  {
    "name": "alpha",
    "status": "active",
    "downloads": 120,
    "version": "1.2.0"
  }
]

```

图 4 实验结果查看

```

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q15 (main)
$ chmod +x api_report.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q15 (main)
$ ./api_report.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q15 (main)
$ ls
api_report.sh*  packages.json  summary.md

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q15 (main)
$ cat summary.md
# Package Summary

| name | version | downloads |
|---|---|---|
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |

```

图 5 实验结果查看

1.4 第 16 题：修复并交付一个陌生的小型工具仓库

主题：课堂综合测试

1.4.1 任务说明

基于第 13 题项目复制创建 q16，模拟实际软件开发中的缺陷修复流程：人为制造问候语输出错误，通过自动化测试定位程序问题；修复代码后重新运行测试，确保程序功能恢复正确；编写 Makefile，实现代码检查、项目构建和清理操作；使用 `python -m build` 生成 wheel 安装包，并计算 SHA-256 校验值完成发布验证。

1.4.2 关键操作

复制 q13 为 q16 并初始化/继续使用 Git；把问候语实现临时改成始终返回字面量 `Hello,name!`，运行 `check.sh` 确认测试失败；随后定位并修复：

制造并定位程序错误：

```
#cli.py
import argparse
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    if not a.name.strip():
        raise SystemExit(2)
    print("Hello, name!") %修改print(f"Hello, {a.name}!")
if __name__ == "__main__":
    main()
```

q16 是新的项目副本，需要重新进行安装；并使用 `pytest` 发现问题：

```
pip install -e .
pytest
```

使用 Makefile 自动化管理；创建 Makefile：

```
.PHONY: check build clean
```

```
check:
```

```
    ./check.sh
```

```
build:
```

```
    python -m build
```

```
clean:
```

```
    rm -rf dist build *.egg-info
```

构建 wheel 并验证发布文件；成功后生成文件：`.whl` 是 Python 二进制安装包、`.tar.gz` 是源码发布包：

```
make build
```

关键点:

- `pip install -e .`: 可编辑安装方式, 使命令入口指向当前源码, 方便开发调试。
- 制造 bug 后通过 `pytest` 失败结果分析错误原因, 再修改源码完成修复。
- `Makefile` 用于自动化管理项目流程, 将检查、构建、清理等操作封装为简单命令。
- `make check` 整合格式检查、静态检查和测试, 提高代码提交前的可靠性。
- `python -m build` 用于生成 Python 发布包, 其中 wheel 文件可通过 `pip` 直接安装。
- SHA-256 校验值用于验证构建产物完整性, 保证发布文件未被修改。

1.4.3 实验结果

```
PS D:\classPai\Class\8.24\q16> pip install -e .
Obtaining file:///D:/classPai/Class/8.24/q16
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: greetlab-25020007046
Building editable for greetlab-25020007046 (pyproject.toml) ... done
Created wheel for greetlab-25020007046: filename=greetlab_25020007046-0.1.0-0.editable-py3-none-any.whl size=1583 sha256=10e83616694b0abfb927b945e9ad05e8ada911a33c4c9bec4216d45ce53d51de
Stored in directory: C:\Users\lx\AppData\Local\Temp\pip-ephem-wheel-cache-1zzrzd28\wheels\fa\fd\64558bca34efd5a3b0d8aa254a21c4ade994b1ae12474901
Successfully built greetlab-25020007046
Installing collected packages: greetlab-25020007046
Attempting uninstall: greetlab-25020007046
Found existing installation: greetlab-25020007046 0.1.0
Uninstalling greetlab-25020007046-0.1.0:
Successfully uninstalled greetlab-25020007046-0.1.0
Successfully installed greetlab-25020007046-0.1.0
WARNING: You are using pip version 22.0.4; however, version 26.0.1 is available.
You should consider upgrading via the 'C:\Users\lx\AppData\Local\Programs\Python\Python39\python.exe -m pip install --upgrade pip' command.
PS D:\classPai\Class\8.24\q16> sdt-greet --name Alice
Hello, name!
```

图 6 错误代码

```
PS D:\classPai\Class\8.24\q16> pytest
===== test session starts =====
platform win32 -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\classPai\Class\8.24\q16
configfile: pytest.ini
plugins: Faker-37.12.0, cov-5.0.0, django-4.11.1
collected 2 items

tests\test_cli.py F. [100%]

===== FAILURES =====
_____ test_normal_name _____

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x00000161115D5BE0>
capsys = <_pytest.capture.CaptureFixture object at 0x00000161115D5CA0>

    def test_normal_name(monkeypatch, capsys):
        monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", "Alice"])

        main()

        captured = capsys.readouterr()

>       assert "Hello, Alice!" in captured.out
E       AssertionError: assert 'Hello, Alice!' in 'Hello, name!\n'
E       + where 'Hello, name!\n' = CaptureResult(out='Hello, name!\n', err='').out

..q13\tests\test_cli.py:12: AssertionError
===== short test summary info =====
FAILED tests\test_cli.py::test_normal_name - AssertionError: assert 'Hello, Alice!' in 'Hello, name!\n'
===== 1 failed, 1 passed in 0.19s =====
```

图 7 错误问候语代码定位

```

PS D:\classPai\Class\8.24\q16> sdt-greet --name Alice
Hello, Alice!
PS D:\classPai\Class\8.24\q16> pytest
===== test session starts =====
platform win32 -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\classPai\Class\8.24\q16
configfile: pytest.ini
plugins: Faker-37.12.0, cov-5.0.0, django-4.11.1
collected 2 items

tests\test_cli.py .. [100%]

===== 2 passed in 0.11s =====
PS D:\classPai\Class\8.24\q16>

```

图 8 测试后修复截图

```

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q16 (main)
$ make check
./check.sh
3 files already formatted
All checks passed!
D:\classPai\Class\8.24\q16\check.sh: line 7: pytes: command not found
make: *** [Makefile:4: check] Error 127

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q16 (main)
$ make build
python -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:

```

图 9 make check 自动测试通过

```

Successfully built greetlab_25020007046-0.1.0.tar.gz and greetlab_25020007046-0.1.0-py3-none-any.whl
lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q16 (main)
$ ls dist
greetlab_25020007046-0.1.0-py3-none-any.whl greetlab_25020007046-0.1.0.tar.gz

```

图 10 Make build 生成发布包

```

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q16 (main)
$ sha256sum dist/*.whl
292c91cab0dad1f28037eef7e3867f3bfff5beeb26cccbdd8afcfcabbc1128c76 *dist/greetlab_25020007046-0.1.0-py3-
none-any.whl

```

图 11 wheel 文件 SHA-256 校验结果

2 课下习题

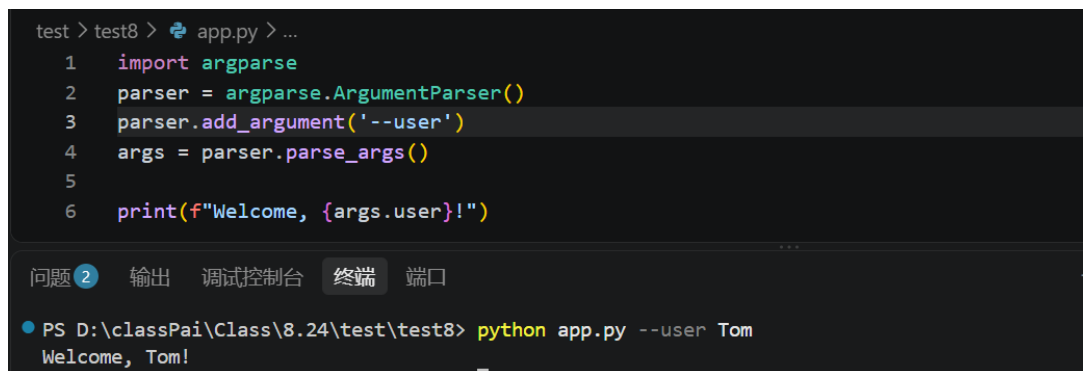
2.1 Python 命令行参数练习

```

1 import argparse
2 parser = argparse.ArgumentParser()
3 parser.add_argument('--user')
4 args = parser.parse_args()
5 print(f"Welcome, {args.user}!")

```

执行 `python app.py --user Tom`
 输出 `Welcome, Tom!`



```
test > test8 > app.py > ...
1 import argparse
2 parser = argparse.ArgumentParser()
3 parser.add_argument('--user')
4 args = parser.parse_args()
5
6 print(f"Welcome, {args.user}!")

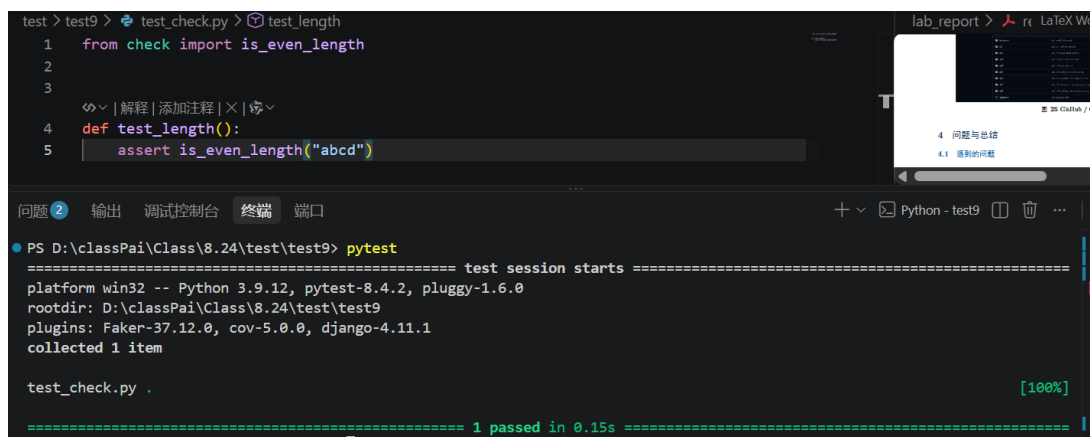
问题 2 输出 调试控制台 终端 端口
PS D:\classPai\Class\8.24\test\test8> python app.py --user Tom
Welcome, Tom!
```

图 12 查看输出

2.2 pytest 测试字符串功能

```
1 %check.py
2 def is_even_length(s):
3     return len(s)%2==0
```

```
1 %test_check.py
2 from check import is_even_length
3 def test_length():
4     assert is_even_length("abcd")
```



```
test > test9 > test_check.py > test_length
1 from check import is_even_length
2
3
4 def test_length():
5     assert is_even_length("abcd")

问题 2 输出 调试控制台 终端 端口
PS D:\classPai\Class\8.24\test\test9> pytest
===== test session starts =====
platform win32 -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\classPai\Class\8.24\test\test9
plugins: Faker-37.12.0, cov-5.0.0, django-4.11.1
collected 1 item

test_check.py . [100%]

===== 1 passed in 0.15s =====
```

图 13 pytest

2.3 使用 ruff 检查代码

错误格式代码:

```
1 def hello():
2     print("hello")
```

输入命令:

```
ruff format --check
ruff format
```

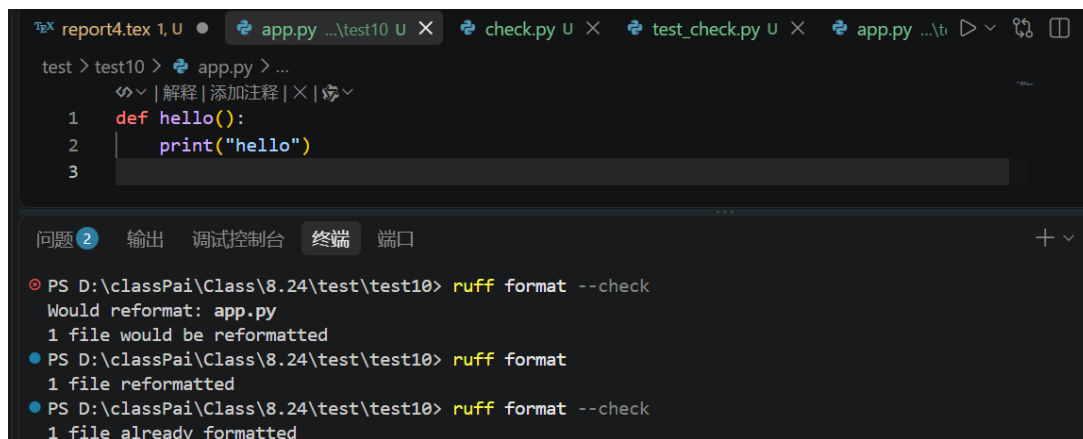


图 14 查看 ruff 结果

2.4 使用 Makefile 管理 Python 程序

```
1 %Makefile
2 .PHONY: run clean
3 run:
4     python main.py
5 clean:
6     rm -f output.txt
```

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/test/test11 (main)
$ make run
python main.py
Hello from main.py!

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/test/test11 (main)
$ ls
Makefile main.py output.txt

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/test/test11 (main)
$ make clean
rm -f output.txt

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/test/test11 (main)
$ ls
Makefile main.py
```

图 15 自动执行结果

2.5 理解 Makefile 依赖关系

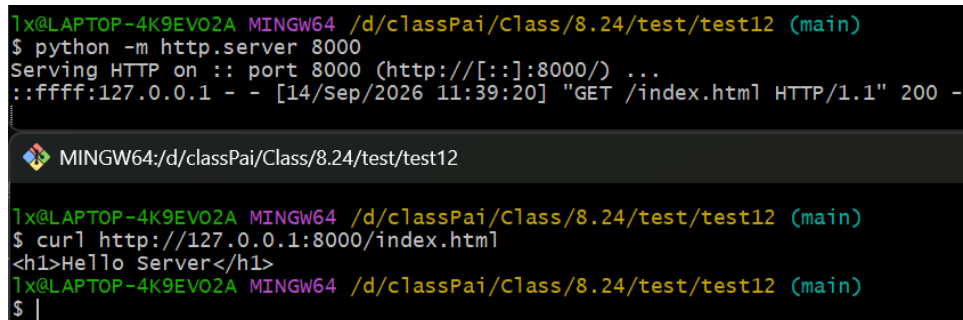
```
%Makefile
result.txt: main.py
python main.py > result.txt
```

```
PS D:\classPai\Class\8.24\test\test11> make
python main.py > result.txt
PS D:\classPai\Class\8.24\test\test11> make
make: 'result.txt' is up to date.
```

图 16 增量构建结果查看

2.6 使用 curl 访问本地服务

```
%index.html
<h1>Hello Server</h1>
```



```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/test/test12 (main)
$ python -m http.server 8000
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
::ffff:127.0.0.1 - - [14/Sep/2026 11:39:20] "GET /index.html HTTP/1.1" 200 -

MINGW64:/d/classPai/Class/8.24/test/test12

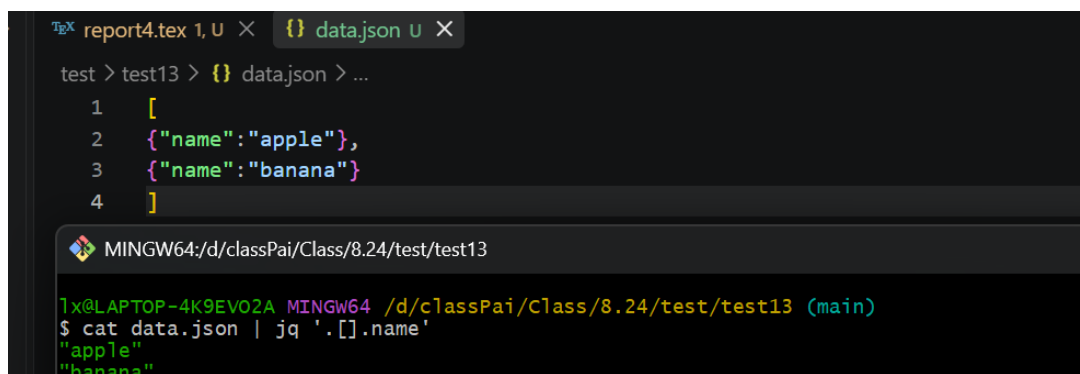
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/test/test12 (main)
$ curl http://127.0.0.1:8000/index.html
<h1>Hello Server</h1>
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/test/test12 (main)
$ |
```

图 17 查看服务器状态

2.7 使用 jq 统计 JSON 数据

```
%data.json
[
  {"name": "apple"},
  {"name": "banana"}
]
```

. 表示当前数据；[] 遍历数组；.name 获取字段。



```
report4.tex 1, U × {} data.json U ×
test > test13 > {} data.json > ...
1  [
2  {"name": "apple"},
3  {"name": "banana"}
4  ]

MINGW64:/d/classPai/Class/8.24/test/test13

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/test/test13 (main)
$ cat data.json | jq '.[].name'
"apple"
"banana"
```

图 18 统计 JSON 文件中所有商品名称

2.8 使用 jq 过滤数据

```
1 %score.json
2 [
3  {"name": "A", "score": 80},
4  {"name": "B", "score": 50}
5  ]
```



```
report4.tex U • {} score.json U X {} data.json U X
test > test14 > {} score.json > ...
1  [
2  {"name": "A", "score": 80},
3  {"name": "B", "score": 50}
4  ]

MINGW64:/d/classPai/Class/8.24/test/test14
1x@LAPTOP-4K9EVO2A MINGW64 /d/c\lassPai/Class/8.24/test/test14 (main)
$ cat score.json | jq '.[] | select(.score>60)'
{
  "name": "A",
  "score": 80
}
1x@LAPTOP-4K9EVO2A MINGW64 /d/c\lassPai/Class/8.24/test/test14 (main)
$ |
```

图 19 查看筛选结果

执行：select() 可以实现条件过滤。

```
cat score.json | jq '.[] | select(.score>60)'
```

2.9 循环执行程序直到失败

```
1 while true; do
2     count=$((count + 1))
3
4     ./random.sh > stdout.log 2> stderr.log
5
6     if [[ $? -ne 0 ]]; then
7         echo "Failed after $count runs"
8         cat stdout.log
9         cat stderr.log
10        break
11    fi
12 done
```

编写脚本循环运行随机失败程序，分别保存标准输出和标准错误。当程序返回非零状态时停止运行，输出失败次数和错误信息。本次实验中程序运行 105 次后失败，成功捕获错误输出。

```
lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ touch random.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ cat > random.sh
#!/usr/bin/env bash

n=$(( RANDOM % 100 ))

if [[ n -eq 42 ]]; then
    echo "Something went wrong"
    >&2 echo "The error was using magic numbers"
    exit 1
fi

echo "Everything went according to plan"

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ chmod +x random.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ls -l
total 1606
-rw-r--r-- 1 lx 197121 1620877 Aug 27 11:58 8.24.docx
-rw-r--r-- 1 lx 197121      65 Sep  3 22:19 marco.sh
drwxr-xr-x 1 lx 197121      0 Aug 26 11:24 q03-git-history-backup/
-rwxr-xr-x 1 lx 197121    205 Sep  3 22:22 random.sh*
-rw-r--r-- 1 lx 197121   15631 Sep  2 11:01 report2.txt
drwxr-xr-x 1 lx 197121      0 Aug 27 23:11 test/

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ touch run_until_fail.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ cat > run_until_fail.sh
#!/usr/bin/env bash

count=0
while true; do
    count=$((count + 1))

    ./random.sh > stdout.log 2> stderr.log

    if [[ $? -ne 0 ]]; then
        echo "Failed after $count runs"
        echo "stdout:"
        cat stdout.log
        echo "stderr:"
        cat stderr.log
        break
    fi
done

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ chmod +x run_until_fail.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ls -l
total 1607
-rw-r--r-- 1 lx 197121 1620877 Aug 27 11:58 8.24.docx
-rw-r--r-- 1 lx 197121      65 Sep  3 22:19 marco.sh
drwxr-xr-x 1 lx 197121      0 Aug 26 11:24 q03-git-history-backup/
-rwxr-xr-x 1 lx 197121    205 Sep  3 22:22 random.sh*
-rw-r--r-- 1 lx 197121   15631 Sep  2 11:01 report2.txt
-rwxr-xr-x 1 lx 197121    301 Sep  3 22:23 run_until_fail.sh*
drwxr-xr-x 1 lx 197121      0 Aug 27 23:11 test/

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ./run_until_fail.sh
Failed after 105 runs
stdout:
Something went wrong
```

图 20 循环执行程序直到失败的结果

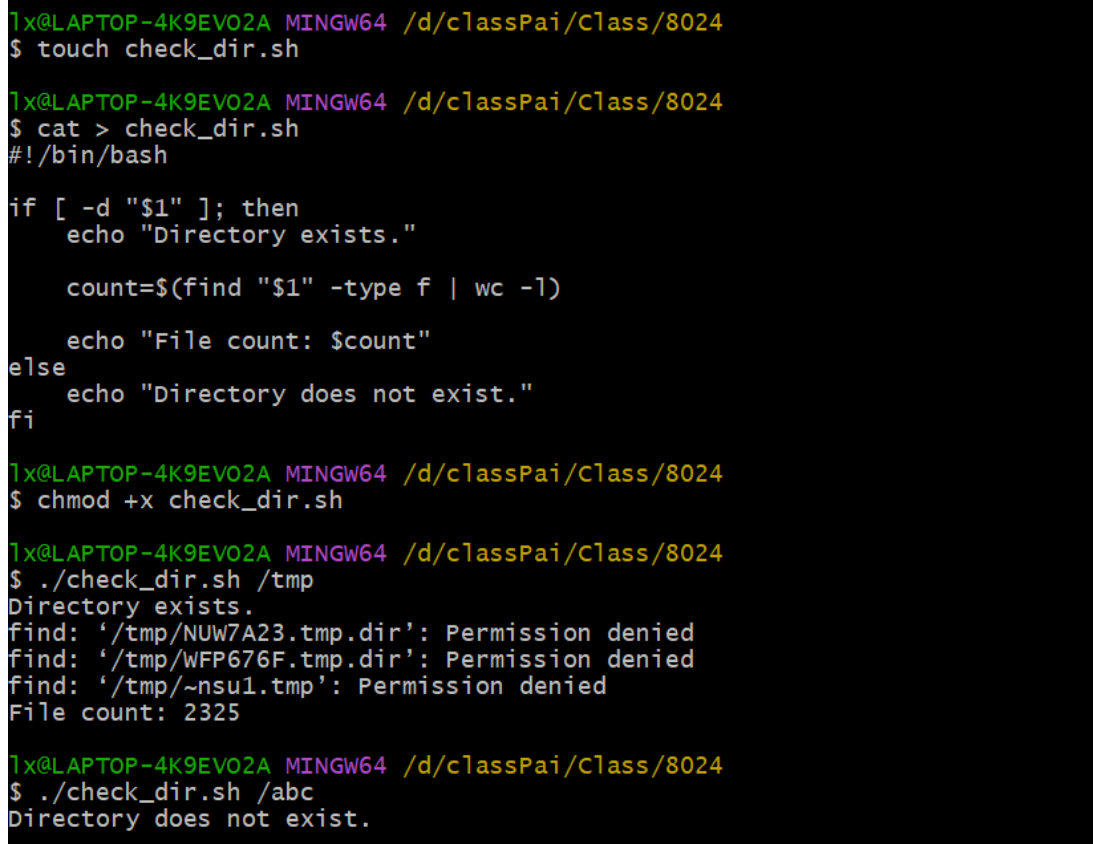
2.10 目录文件数量统计脚本

编写 Shell 脚本 `check_dir.sh`，通过输入目录路径判断目录是否存在，并统计其中普通文件数量。

```
1 #!/bin/bash
2
3 if [ -d "$1" ]; then
4     echo "Directory exists."
5
6     count=$(find "$1" -type f | wc -l)
7
8     echo "File count: $count"
9 else
10     echo "Directory does not exist."
11 fi
```

其中，`$1` 表示脚本输入的的第一个参数，`-d` 用于判断目录是否存在，`find` 命令用于查找普通文件，`wc -l` 用于统计文件数量。

测试结果显示，当输入存在目录时，脚本能够输出文件数量；输入不存在目录时，能够正确提示目录不存在。



```
lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ touch check_dir.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ cat > check_dir.sh
#!/bin/bash

if [ -d "$1" ]; then
    echo "Directory exists."

    count=$(find "$1" -type f | wc -l)

    echo "File count: $count"
else
    echo "Directory does not exist."
fi

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ chmod +x check_dir.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ./check_dir.sh /tmp
Directory exists.
find: '/tmp/NUW7A23.tmp.dir': Permission denied
find: '/tmp/WFP676F.tmp.dir': Permission denied
find: '/tmp/~nsul.tmp': Permission denied
File count: 2325

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ./check_dir.sh /abc
Directory does not exist.
```

图 21 目录文件数量统计脚本运行结果

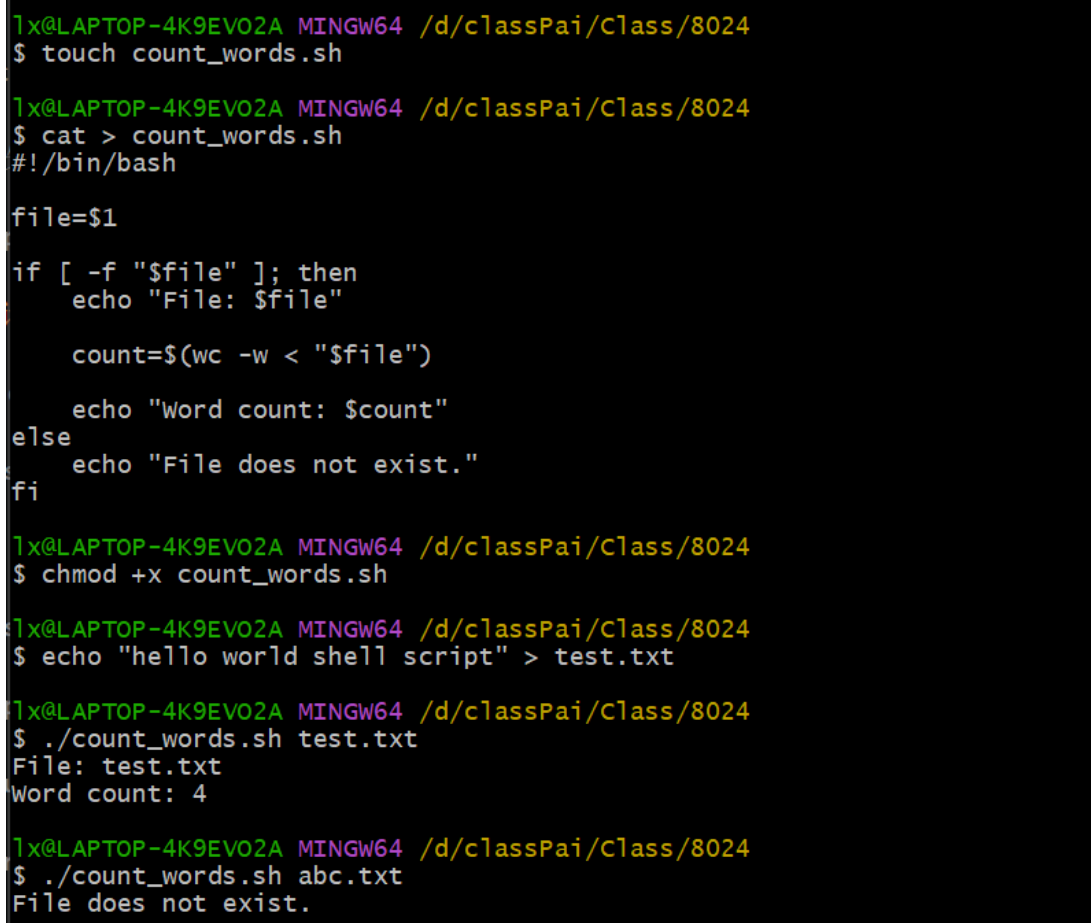
2.11 文本文件单词数量统计脚本

编写 Shell 脚本 `count_words.sh`，实现输入文本文件路径并统计文件中单词数量的功能。

```
1  #!/bin/bash
2
3  file=$1
4
5  if [ -f "$file" ]; then
6      echo "File: $file"
7
8      count=$(wc -w < "$file")
9
10     echo "Word count: $count"
11 else
12     echo "File does not exist."
13 fi
```

脚本通过 `$1` 获取输入文件路径，使用 `-f` 判断文件是否存在，并利用 `wc -w` 命令统计文件中的单词数量。

测试结果表明，当输入有效文件时，脚本能够正确输出单词数量；当输入不存在的文件时，能够给出错误提示。



```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ touch count_words.sh

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ cat > count_words.sh
#!/bin/bash

file=$1

if [ -f "$file" ]; then
    echo "File: $file"

    count=$(wc -w < "$file")

    echo "Word count: $count"
else
    echo "File does not exist."
fi

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ chmod +x count_words.sh

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ echo "hello world shell script" > test.txt

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ./count_words.sh test.txt
File: test.txt
Word count: 4

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ./count_words.sh abc.txt
File does not exist.
```

图 22 统计文本文件单词数量脚本运行结果

3 Git 与作业提交

3.1 仓库内容整理

```
8.24/  
|-- q01/  
|-- q02/  
|-- q03/  
|-- q04/  
|-- q05/  
|-- q06/  
|-- q07/  
|-- q08/  
|-- q09/  
|-- q10/  
|-- q11/  
|-- q12/  
|-- q13/  
|-- q14  
|-- q15  
|-- q16  
`-- lab_report/  
    |-- figures/  
    |-- report.tex  
    |-- report.pdf  
    |-- report2.tex  
    |-- report2.pdf  
    |-- report3.pdf  
    |-- report3.tex  
    |-- report4.pdf  
    `-- report4.tex
```

3.2 GitHub / Gitee 提交记录

仓库地址: <https://github.com/haru-senya/8.24.git>

3.3 commit 记录截图、仓库页面截图

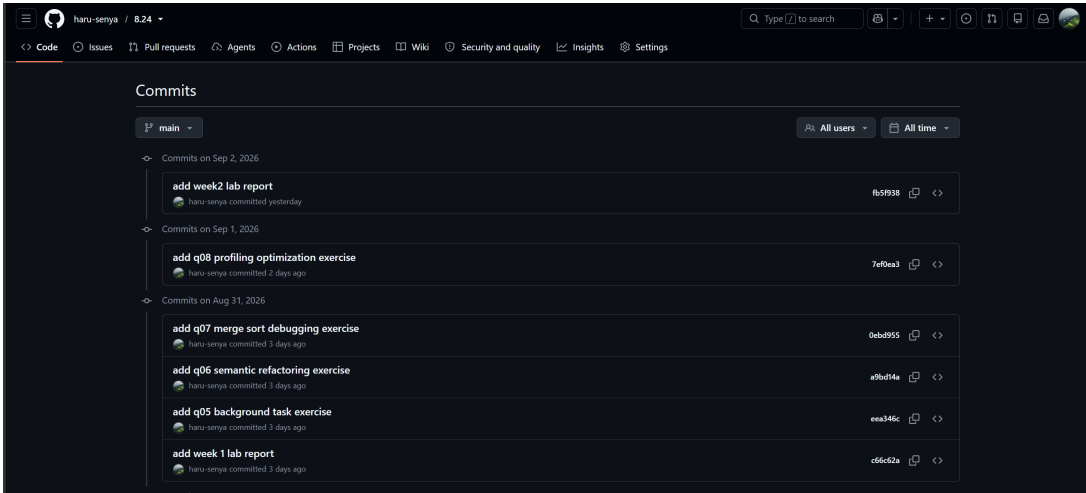


图 23 Git commit 记录

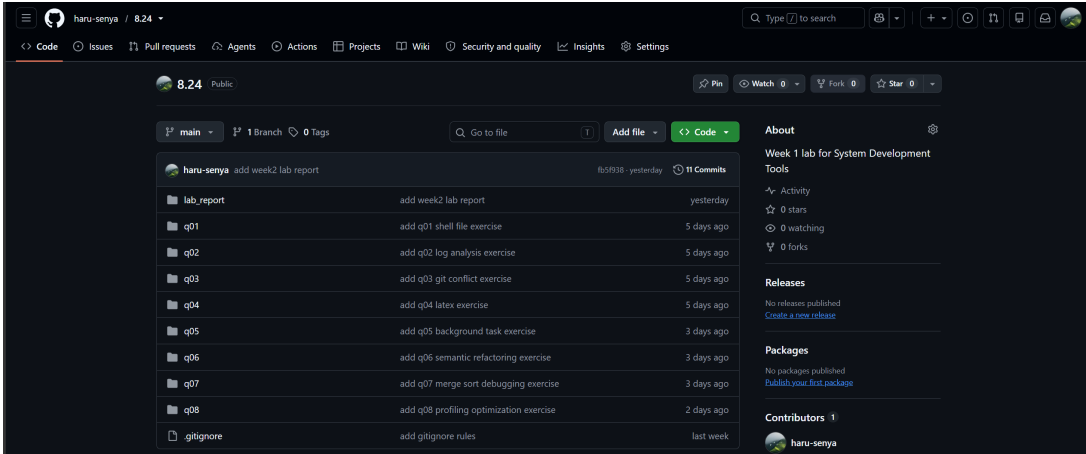


图 24 GitHub / Gitee 仓库页面

4 问题与总结

4.1 遇到的问题

- 命令入口无法识别：第 13 题配置 `pyproject.toml` 后直接运行 `sdt-greet --name Alice`，出现 `command not found`。原因是项目虽然配置了脚本入口，但当前环境没有安装该项目。通过执行 `pip install .` 安装项目，使系统生成对应命令入口，之后可以正常调用 `sdt-greet`。
- 修改源码后命令仍执行旧版本：第 16 题从 q13 复制项目到 q16 后，修改了源码中的问候语输出，但运行 `sdt-greet` 仍得到旧结果。原因是之前安装的命令入口仍指向旧项目。使用 `pip install -e .` 进行可编辑安装，使命令入口重新绑定当前源码目录，修改后的代码能够及时生效。
- `pytest` 无法通过命令执行：第 13、16 题运行检查脚本时出现 `pytest: command not found`。检查发现 Python 环境中已经安装 `pytest`，但终端无法直接找到对应命令。将脚本中的 `pytest` 修改为 `python -m pytest`，通过 Python 模块方式调用测试工具，提高了环境兼容性。

- **ruff 格式检查失败**：第 13 题第一次运行 `./check.sh` 时，ruff 提示部分 Python 文件需要重新格式化。使用 `ruff format` 自动调整代码格式，再运行 `ruff format --check` 验证，最终所有文件通过格式检查。
- **Makefile 命令执行逻辑不理解**：第 14 题编写 Makefile 时，对 `make`、目标 (target) 以及依赖关系理解不足。通过分析 Makefile 中的规则，例如 `check` 调用检查脚本，`build` 调用构建命令，理解了 Makefile 可以将多个重复操作封装成简单命令，提高项目管理效率。
- **curl 无法连接本地服务**：

```
curl[http://127.0.0.1:8000/packages.json](http://127.0.0.1:8000/packages.json)
```

时提示无法连接服务器。检查发现本地 HTTP 服务没有启动。使用 `python -m http.server 8000` 启动服务后，curl 能够正常获取 JSON 数据。
- **理解 jq 筛选与排序逻辑困难**：第 15 题中需要筛选 `active` 状态且 `downloads` 不少于 100 的数据，同时按照下载量降序、名称升序排列。通过学习 `.[ ]` 遍历数组、`select()` 条件过滤以及 `sort_by()` 排序，理解了命令行工具处理结构化数据的流程。
- **SHA-256 校验作用不理解**：第 16 题计算 wheel 文件哈希值时，不清楚随机字符串的意义。通过了解 SHA-256 后认识到其相当于文件数字指纹，可以用于验证发布文件是否被修改，保证软件交付完整性。

4.2 实验总结

本周实验综合实践了软件开发的完整工程流程，涵盖代码修改、自动化测试、项目构建与发布验证。通过人为制造问候语输出错误，体验了测试驱动的问题定位过程，认识到测试不仅能证明程序正确，还能帮助快速发现逻辑错误。实验中进一步学习了 Makefile 自动化工具，将检查、构建等重复操作封装为简单命令，提升了项目管理效率；同时通过 wheel 构建和 SHA-256 校验，了解了 Python 项目从源码开发到软件交付的基本流程。过程中遇到了 Python 包安装路径、命令入口更新、pytest 环境配置等问题，通过查阅报错、分析运行流程并修改配置，逐渐理清了开发环境、测试工具与构建工具之间的关系。经过最后一周实验，对软件工程中“开发—测试—修复—构建—发布”的完整流程有了更系统的认识，也提升了运用 Git、Python 工具链及自动化脚本解决实际问题的能力。